

Byte Flow: Byte Facility Software Report

Benjamin Zadorian : Team Leader

Ehsan Nawrouzi : Developer 1

Ahmed Merza : Secretary

Razvan Moinea : Developer 2

Dara Shali : Tester 1

Jakub Zawadzki : Tester 2

Introduction.....	2
Design.....	2
Data Structures & Algorithms.....	2
UI/UX Design	6
Testing	7
Conclusion.....	8
References	9

Introduction

For this project brief we have designed a software application that swimming pool facilities can use to provide services to customers such as registration, lesson booking, renting equipment, purchasing merchandise, renting lockers and more. It also provides services to the staff of these facilities, such as tracking inventory, viewing employee statistics, scheduling lessons and viewing related lesson information. The services that the software provides are distinguished by user authentication and validation to see if the login credentials given are either employee or customer.

This report will firstly cover the design decisions behind the project, why we chose certain technologies, data structures, algorithms and UX design. It will then cover the testing parameters that we put the software through and analysis of the data structures and algorithms. The report will then conclude by giving a summary of the work that we completed, any limitations and obstacles that we faced, and what we would do if we were to reproduce this project knowing what we know now. The final section of the report is a collection of Harvard references with in-text citations alongside the references.

Design

Data Structures & Algorithms

Data Structure

For our project we chose to use a hash table to store employee information that needs to be readily accessible for the user, such what employee is assigned to what pool and when employees are booked for lessons. A hash table is more suitable than a hash map for this project as hash tables do not accept null keys and values. We will hold employee information as this can be retrieved from an SQL table, then stored in the hash table so that it can be retrieved faster and changed when a user books a lesson or changes their membership.

The hash table itself is created using generics with a TKey and TValue generics so that we can use the same hash table structure for different object types such as storing customer and employee information in two different tables.

Hash Table Time Complexity

A hash table was chosen as they are a very fast and flexible way of storing data, with a worst case for 'space', 'insert', 'lookup' and 'delete' of $O(n)$ ([Interview Cake: Programming Interview Questions and Tips, n.d.](#)).

Algorithm Used & Justification

The hashing algorithm we will use is Fowler-Noll-Vo (FNV-1A) ([Wikipedia Contributors, 2019](#)). I chose this algorithm because it is a simple to implement algorithm and has been tried and tested over many years to be a secure hash function. We chose this over FNV-1 because FNV-1A has a better avalanche effect. This is important because the avalanche effect refers to how drastically

the hash result will change based on the smallest of changes to the input, the more that changes based on the input the better as this will lead to a lower chance of collisions ([Wikipedia Contributors, 2020](#)).

The reason we chose to use FNV-1A compared to any other hashing algorithm such as SHA-1 or another SHA derivatives is that, if the input that you try to hash is just 0's, the hash key will always return a 0 as you will be using multiplication on 0, which is 0. As FNV-1A uses a hash offset it removes this error as there is no multiplication by 0 ([Wikipedia Contributors, 2019](#)). This is also useful as it works to reduce the number of possible collisions.

FNv-1A Pseudocode:

```
fnv-1A is
    hash = FNV_offset_basis
    for each byte of data to be hashed do
        hash = hash XOR byte_of_data
        hash = hash × FNV_prime
    return hash
```

FNv-1A Time & Space Complexity

The algorithm that we will be using for hashing is FNV-1A, as this algorithm takes each single byte of data from an input, XOR's that with a hash value that is set to a unsigned long integer, then multiplies that by a unsigned long integer that is prime, this algorithm is $O(n)$ time complexity, where n is the length of the input.

The space complexity can be calculated by adding together all the memory requirements of all variables used in the method ([baeldung, 2020](#)), and finding the highest order of n . For this algorithm we have variables that include:

- Input: $1n$ bytes
- MAX_INPUT: 4 bytes
- FNV_OFFSET_BASIS: 8 bytes
- Hash_value: 8 bytes
- C: 1 byte
- FNV_PRIME: 8 bytes
- TABLE_SIZE: 4 bytes
- Result: 8 bytes
- Return value: 4 bytes

This means that the memory usage of this algorithm is: $1n + 4 + 8 + 8 + 1 + 8 + 4 + 8 + 4$. As $1n$ is the highest order of n , the space complexity is $O(n)$.

Collision Handling: Double Hashing

For handling collisions within the hash table, we have decided to use double hashing, the goal of double hashing is to create a scatter after a collision. This is done by using a second hash of the input as an offset for placing the data, if a collision occurs ([Wikipedia Contributors, 2023](#)). The main point of double hashing is that values that map to the same hash index will get a different offset.

This is done by using a prime number that is smaller than the table size. This algorithm uses mod, because of this we must not get 0 as a result, as adding this to another index results in the same index. To avoid getting 0, you can either add 1 to the result of the second hash or subtract the result of the second hash from the prime number.

Double Hashing Pseudocode:

For double hashing, there are two algorithms that we use. As double hashing requires a prime number less than the table size, we use the Sieve of Eratosthenes algorithm to find the largest prime number less than the table size ([Wikipedia Contributors, 2025](#)). The second algorithm is the double hashing algorithm itself. The sieve of Eratosthenes is passed to the double hashing function to get the `DOUBLE_HASH_PRIME` value.

Double-Hashing is

```
hash = 0
for each byte of data to be hashed do
    hash = hash mod DOUBLE_HASH_PRIME
return hash + 1
```

Sieve of Eratosthenes Pseudocode:

sieve-of-Eratosthenes is

```
prime_array = Boolean[TABLE_SIZE]

for each element in prime_array do
    element = true

for each element in prime_array starting from 2 do
    if element at position - 1 == true
        every multiple of that element = false
for each element in prime_array starting from the last element do
    if element == true
        return element

if no prime is found
```

```
return -1
```

PBKDF2 Algorithm

I used the PBKDF2 algorithm to create the hash and salt for the passwords of users when they register and update their passwords. The salt and hash are then saved in the SQL database and de-crypted when the user logs in. It is decrypted by reversing the encryption method on the hashed password using the salt, and comparing the de-crypted password to the new password that is inputted. I used this algorithm as it is specifically used for passwords as it is computationally intensive. ([Wikipedia Contributors, 2025](#))

PBKDF2 Pseudocode:

FUNCTION HashPassword(password):

```
    salt = GenerateRandomBytes(16)
```

```
    hash = PBKDF2(password, salt, HMACSHA256, 100000, 32)
```

```
    RETURN (salt, Base64Encode(hash))
```

PBKDF2 Time & Space Complexity

The time complexity of this algorithm is tied to three factors, the number of chosen iterations, 100,000 in my case, the password and the salt. This makes the PBKDF2 algorithm have a time complexity of $O(n)$, where n is the number of iterations. The more iterations the harder it would be to brute-force the password hash. This algorithm has a time complexity of $O(1)$, as the only space that is taken up is the size of the key, which is 32 bytes.

Time Tests for HashTable

Function Name	Input Size	Average Time Taken (ms)
Hash 1	5	0.2269
Hash 1	50	0.2469
Hash 1	256	0.2221
Hash 2	5	0.1154
Hash 2	50	0.0913
Hash 2	256	0.0912

Sieve of Eratosthenes	16	0.0049
Sieve of Eratosthenes	100	0.0064
Sieve of Eratosthenes	256	0.0086
Fill Table	16	0.8077
Fill Table	100	0.7081
Fill Table	256	1.1259
Delete Table	16	0.3813
Delete Table	100	0.3907
Delete Table	256	0.4315
Test Insert	16	0.6631
Test Insert	100	0.6661
Test Insert	256	0.7047
TestLookUp	16	0.3720
TestLookUp	100	0.2442
TestLookUp	256	0.3278
TestDelete	16	0.2056
TestDelete	100	0.3401
TestDelete	256	0.3102
TestIsFull	16	0.0002
TestIsFull	100	0.0011
TestIsFull	256	0.0002

Collision Tests for Hash Table

Collision Test: Fill Table	16	45
Collision Test: Fill Table	100	307
Collision Test: Fill Table	256	4422

UI/UX Design

In doing research for our project, I found that many other pieces of software use similar colors such as bright greens, reds and blues. The most prominent example being a software package called EZFacility ([EZFacility, 2025](#)). This facility software was a good piece of inspiration when it came to designing our frontend as it had a simple and sleek design that didn't crowd the user with information.

Layout and Navigation

For the layout of our software, we used a navigation bar on the side of screen that allowed the user to access all pages from the home page. We also used buttons inside of pages themselves that link together. When designing these navigation elements, I used different designs depending on the user interaction that I wanted, if there was a button to purchase a piece of equipment or membership, I used a filled colored button, this draws the users attention to that button making them more likely to make a purchase. For buttons that canceled purchases or lessons, I designed

those buttons to not stand out from the background, this use of dark design patterns can help secure more user purchases from customers.

Testing

Testing Approach

In the project, I created multiple files of unit test cases that validate the functionality of my service files that are to be injected into my front-end pages. These tests included CRUD operation testing and other, more specialized testing that related to that service only.

Testing Environment

I used .NET 9.0.3 with Blazor, and the testing library I used was xUnit.

Test Cases for Services

Category	Number of Tests	Use Case	Output
Record Creation	12	To create a customer, employee or other object for the facility	Return True, record created
Record Deletion	20	Delete an object from the SQL database	Return true, record deleted
Record Retrieval	18	Retrieve data from the SQL database	Return object of record
Record Update	29	Update and edit a record within the SQL database	Return object of record
Employee Teaching Status	2	Retrieve the boolean value of if a record within the EmployeesTable is teaching or not	Return Boolean, depending on if the employee is teaching or not
Customer Membership Status	3	Retrieve the boolean value of if a record within the CustomersTable is a member or not	Return Boolean, depending on if the customer is a membership or not
Customer Membership Duration Test	2	Retrieve the TimeSpan object of how long a customer's membership is valid for.	Return TimeSpan object

Error Handling Tests	49	Used to test and validate SQL functions and EF Core connection methods	Return false/null, no records functions take place
Facility Open Time Test	1	Return a TimeSpan object of how long a record within the FacilitiesTable is open for.	Return TimeSpan object
Locker Assign Owner Tests	2	Update a value within the LockersTable and create a foreign key relation to a record in the CustomersTable	Return customer object created from the record updated
Order Get Latest Order Test	3	Retrieve the most recent record created within the OrdersTable	Return object of latest order record
Entity Relationship Handling Tests	11	Used to test if the relationship between two objects, records or entities is successful and valid	Return true if relationship is connected, false otherwise

Not all tests are present to fit within 7-page limitation, a collection of all unit tests is located within the software code repository.

Conclusion

The development of this project has been a great learning experience for myself as I have learnt much more about EF Core, SQL and creating more developed C# applications. I have also learnt about the structure of a team in the software development field and the issues that can come along with that. As I started working on the backend first, I believe that the backend of our project is much stronger than the front-end. This is because I could easily update and iterate on the custom hash table, and what functions they used, whilst also having time to understand properly how a hash table really works. I also learnt about the unit testing libraries and how to layout unit tests.

It is clear to me that there are certainly some limitations within the project, the main being the UI/UX design as this was the part of the project, I tackled last as getting all the functionality working was more important. There are also some limitations with some specific cases of functionality, such as setting individual shifts for individual days as an employee. All these limitations are caused by no one else other than the team leader contributing anything to the project during the 12-week time period. Another cause is also the limited knowledge of SQL and EF Core, making the tables that I created early into the project harder to modify and work with, which is something I would change if I was to re-create this project.

If I was to re-do this project, I would change my approach to SQL and EF Core as that is the part of this project that took the longest for me to fully understand, as I would run into issues with adding

migrations and updating the database anytime I wanted to change a field or add a foreign key relationship to another table. I would also have a clearer idea how the pages would be designed from the start so that I know what features need to be added, this is also to say that I would conduct more market research before starting right away.

References

1. Interview Cake: Programming Interview Questions and Tips. (n.d.). Hash Table/Hash Map Data Structure | Interview Cake. [online] Available at: <https://www.interviewcake.com/concept/java/hash-map>
2. Wikipedia Contributors (2019). Fowler–Noll–Vo hash function. [online] Wikipedia. Available at: https://en.wikipedia.org/wiki/Fowler-Noll-Vo_hash_function
3. Wikipedia. (2020). Avalanche effect. [online] Available at: https://en.wikipedia.org/wiki/Avalanche_effect
4. Baeldung (2020). Space Complexity | Baeldung on Computer Science. [online] www.baeldung.com. Available at: <https://www.baeldung.com/cs/space-complexity>
5. Wikipedia. (2023). Double hashing. [online] Available at: https://en.wikipedia.org/wiki/Double_hashing
6. Wikipedia. (2025). Sieve of Eratosthenes. [online] Available at: https://en.wikipedia.org/wiki/Sieve_of_Eratosthenesw
7. Wikipedia. (2025). PBKDF2.[online] Available at: <https://en.wikipedia.org/wiki/PBKDF2>
8. EZFacility. (2025). Scheduling, Management & Membership Software | EZFacility. [online] Available at: <https://www.ezfacility.co.uk/> [Accessed 18 Apr. 2025].